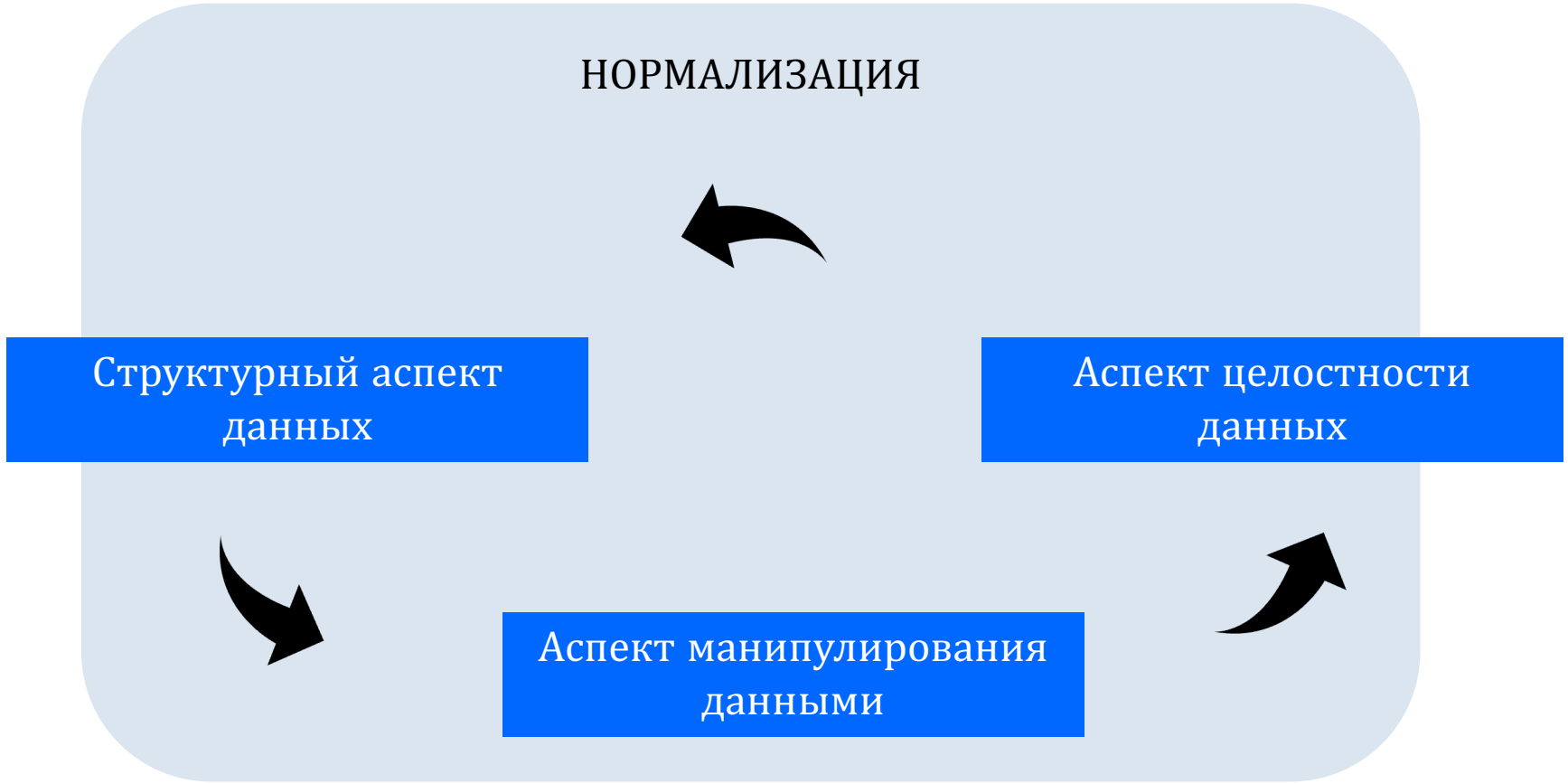
Реляционная алгебра. Теоретико-множественные операции

ст.преподаватель кафедры АСУ
Валова Анастасия Александровна

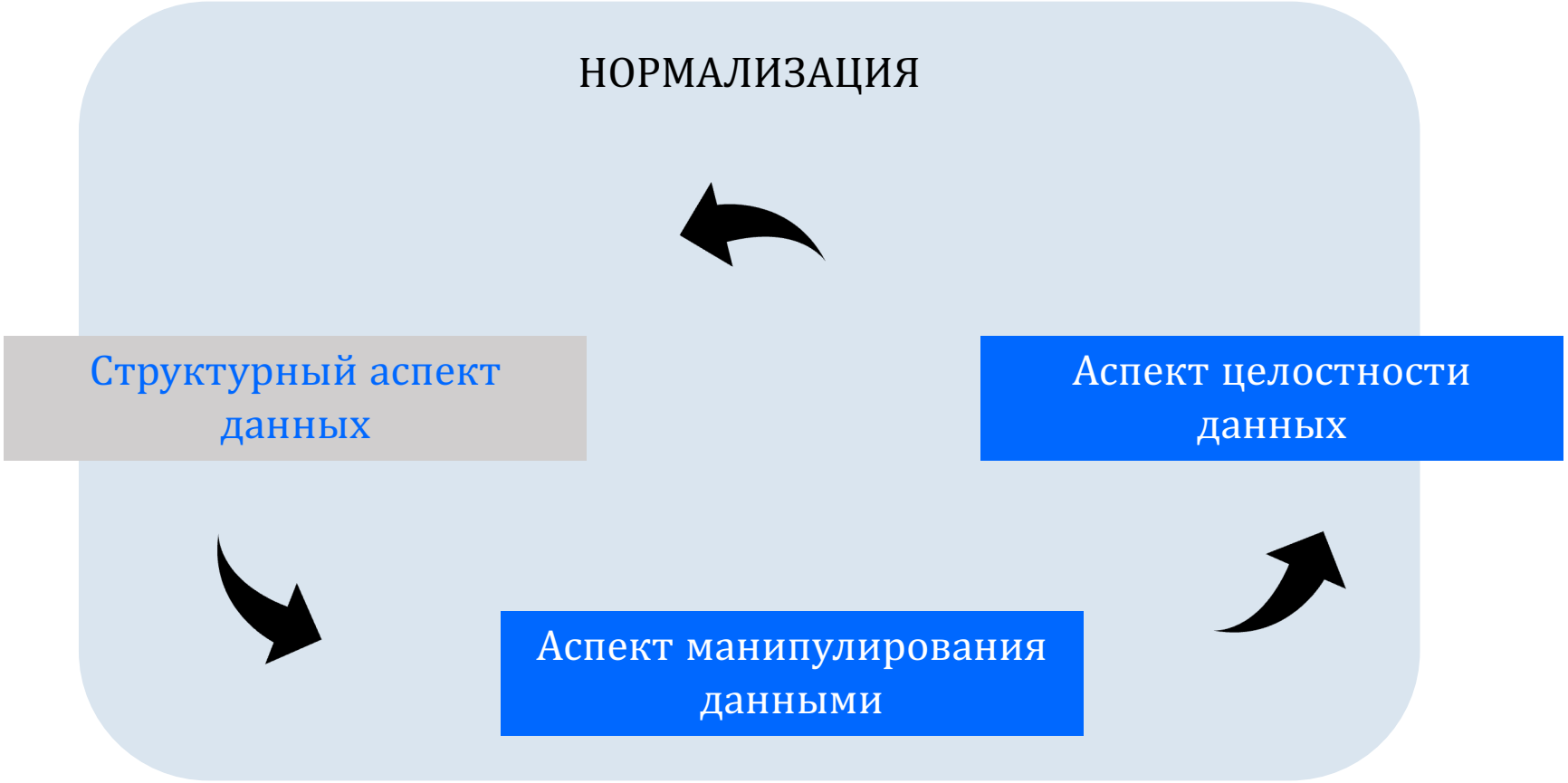
План

- Концепция реляционной модели данных (РМД)
- Компоненты РМД
- Структура данных и понятие «отношения»
- Первичный и внешний ключ отношения
- Ограничения целостности
- Операции реляционной алгебры
- Подготовка к лабораторным работам № 3,4

Реляционная модель данных



Реляционная модель данных



Структурный аспект
Структура данных в РМД

Домены:
ID (int)
Name (varchar)
City (varchar)
Age (int)

Атрибуты

Первичный ключ (PK)

ID	Name	City	Age
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	28

Кортеж

Отношение A

Структурный аспект
Понятие «отношения»

$$T = \left\{ T_i ; i = \overline{1, n} \right\}$$ - множество доменов

Отношение (R)

$$R = \left\langle H; t = \left\{ t_j ; j = \overline{1, m} \right\} \right\rangle$$ степень отношения= |H|, кардинальность отношения = |t|

$$H = \left\{ A_i : T_i ; i = \overline{1, n} \right\}$$ - заголовок (множество атрибутов)

$$t_j = \left\{ A_i : v_i ; i = \overline{1, n} \right\}$$ - тело (множество кортежей)

Структурный аспект
Первичный ключ (Primary key)

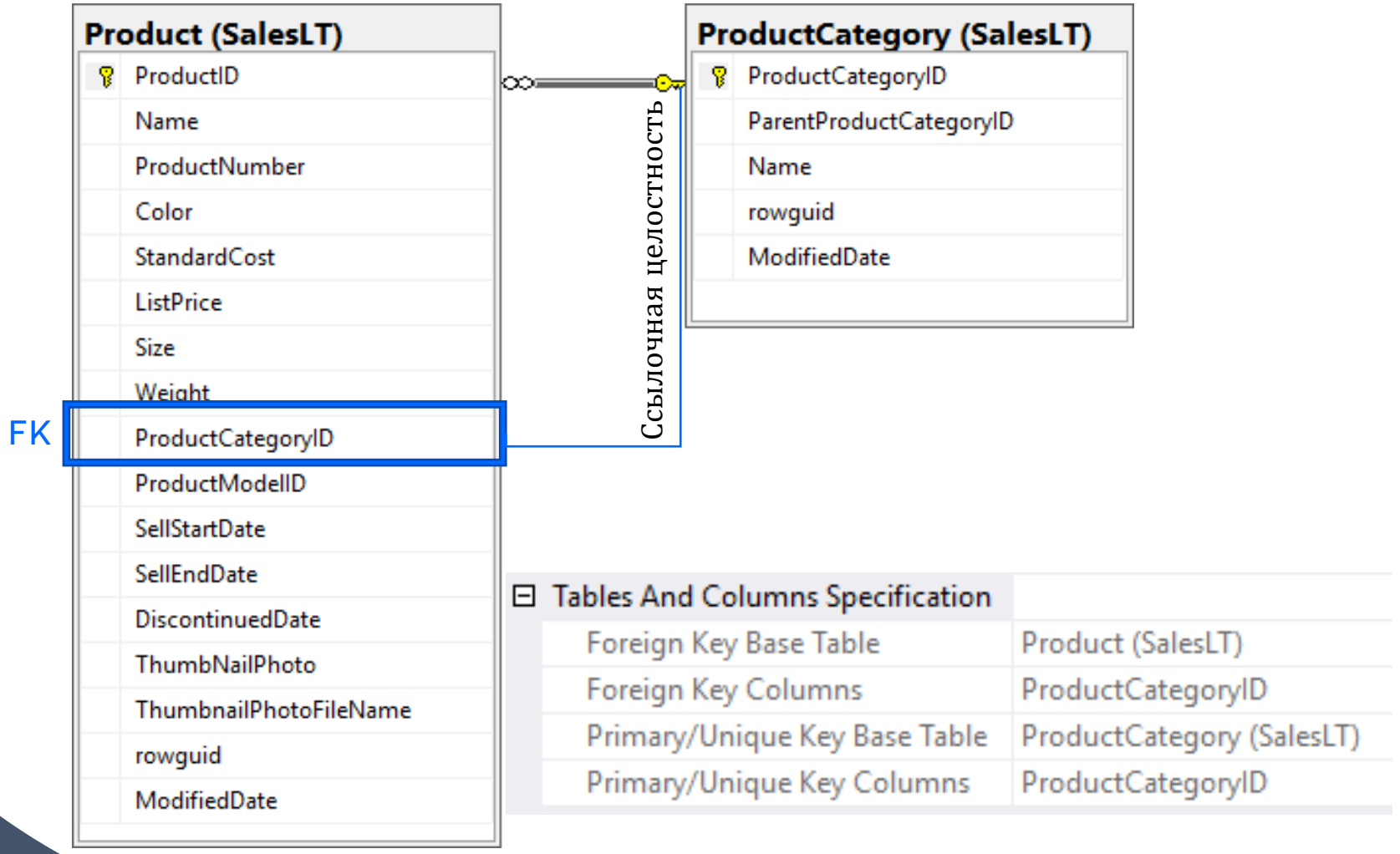
Первичный ключ - один домен (или комбинация доменов) отношения содержащий значения, позволяющее однозначно идентифицировать каждый элемент (n-кортеж) этого отношения.

Потенциальный первичный ключ

Подмножество атрибутов отношения, удовлетворяющее требованиям:

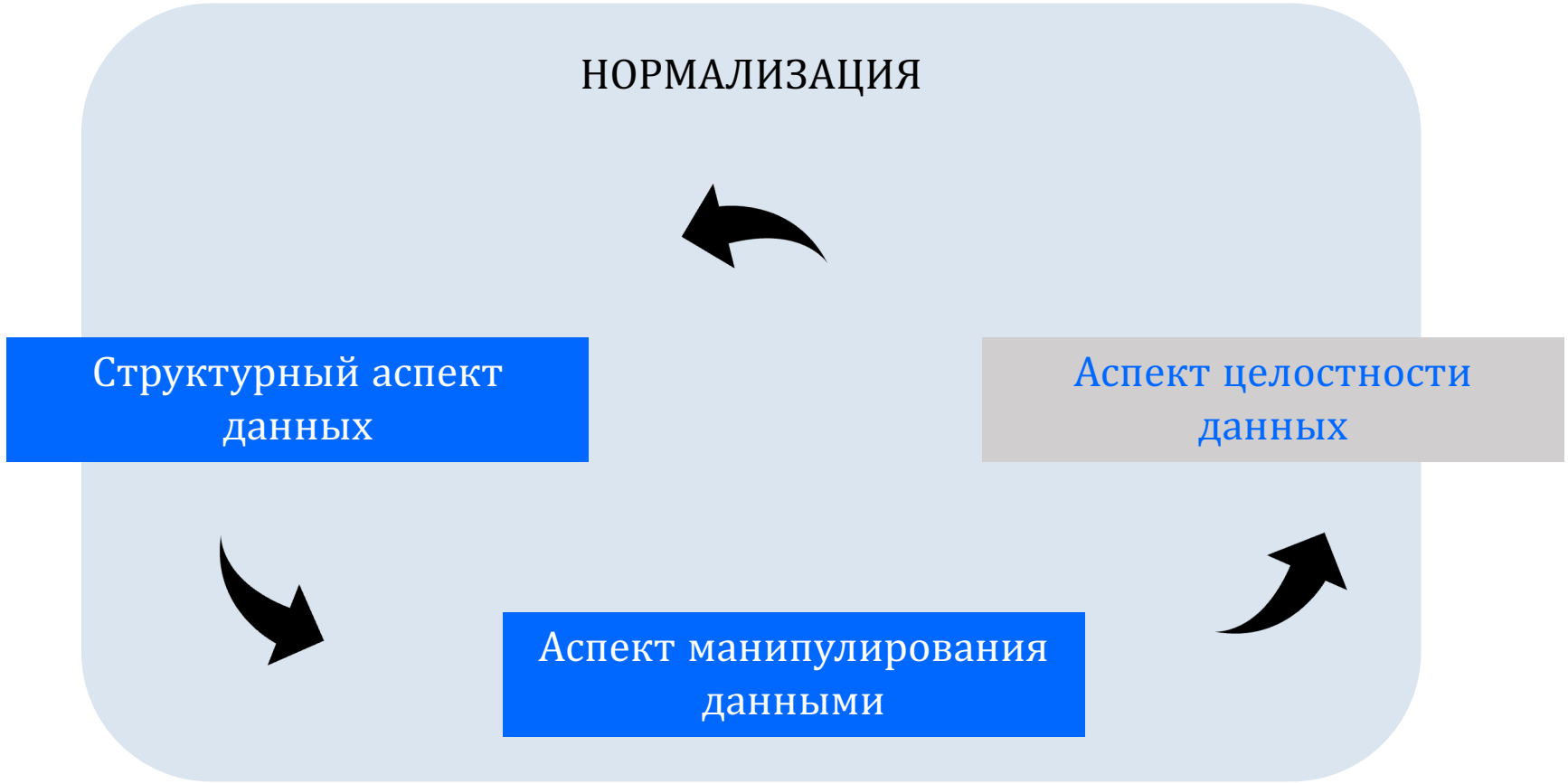
- *Уникальность* - нет (и не может быть) двух кортежей, в которых значения этого подмножества атрибутов совпадают (равны);
- *Неизбыточность* - отсутствует подмножество атрибутов потенциального ключа удовлетворяющее условию уникальности.

Структурный аспект
Внешний ключ (Foreign key)



Tables And Columns Specification	
Foreign Key Base Table	Product (SalesLT)
Foreign Key Columns	ProductCategoryID
Primary/Unique Key Base Table	ProductCategory (SalesLT)
Primary/Unique Key Columns	ProductCategoryID

Реляционная модель данных



Аспект целостности данных

Классификация целостных ограничений

- ***Ограничения типа (домена)*** – перечень допустимых значений типа

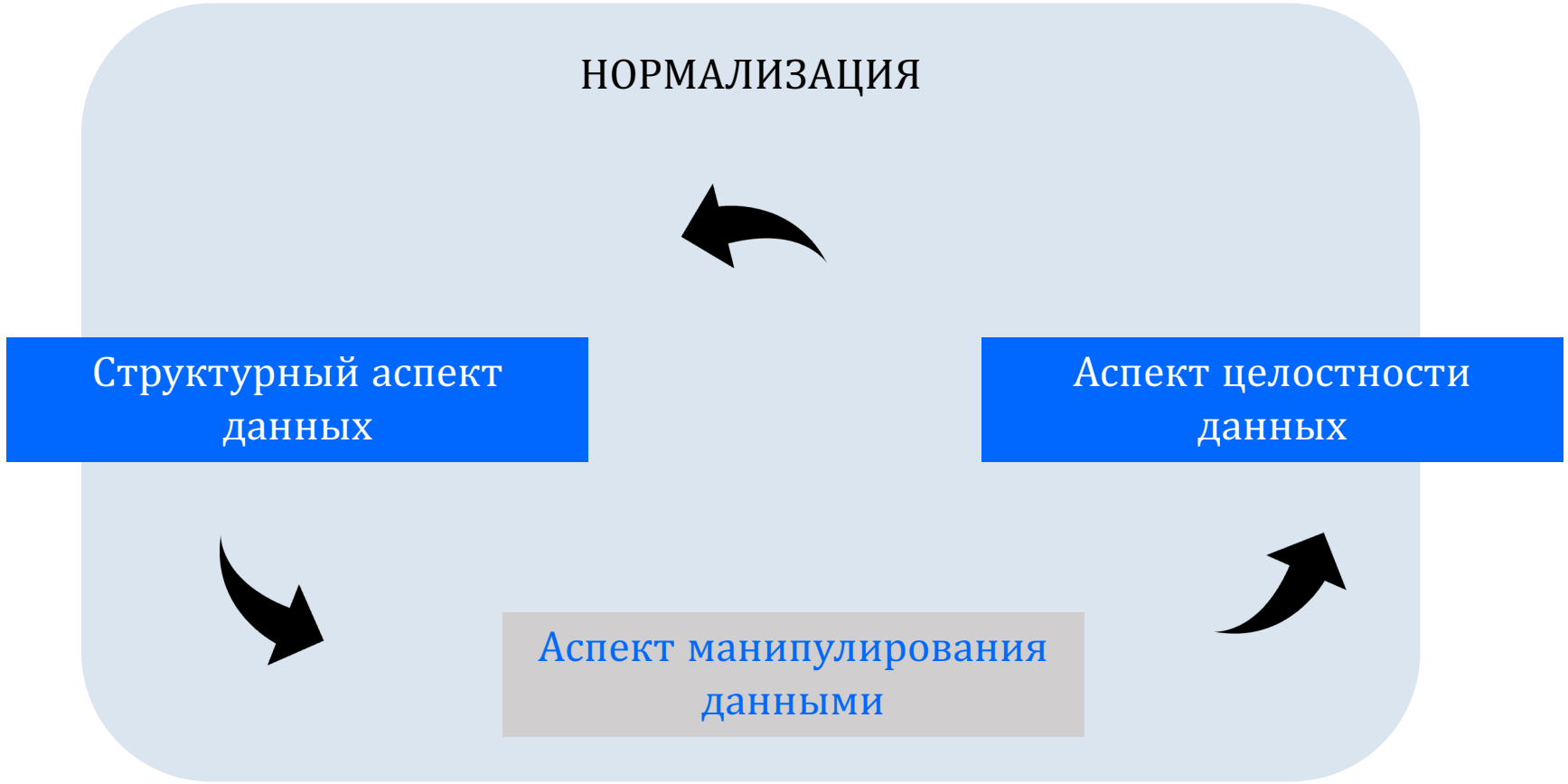
ID (int)	FirstName (varchar)
первый	Иванов

- ***Ограничения атрибута*** – объявление о том, что определенный атрибут имеет определенный тип

ID	Phone
1	+7 123 456 78 --

- ***Ограничения отношения*** – допустимые значения для данного отношения
- ***Ограничения базы данных*** – взаимосвязи между отношениями

Реляционная модель данных



Аспект манипулирования данными
Операции реляционной алгебры

Операции реляционной алгебры из теории множеств

- Объединение (UNION) (∪),
- Пересечение (INTERSECTION) (∩),
- Разность (DIFFERENCE) (\),
- Декартовое произведение (CARTESIAN PRODUCT) (×)

Унарные операции реляционной алгебры

- Проекция
- Выборка

Бинарные операции реляционной алгебры

- Деление (DIVISION)
- Соединения (JOIN)

Реляционная алгебра
Объединение

Отношение A

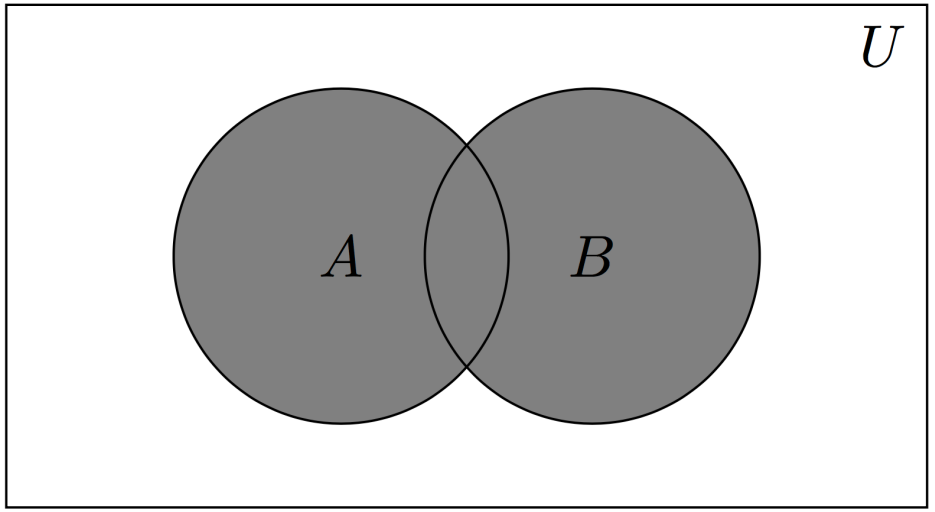
ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21

Отношение B

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1896	Галкин	Иваново	40

A ∪ B

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21
1896	Галкин	Иваново	40



Реляционная алгебра

Пересечение

Отношение A

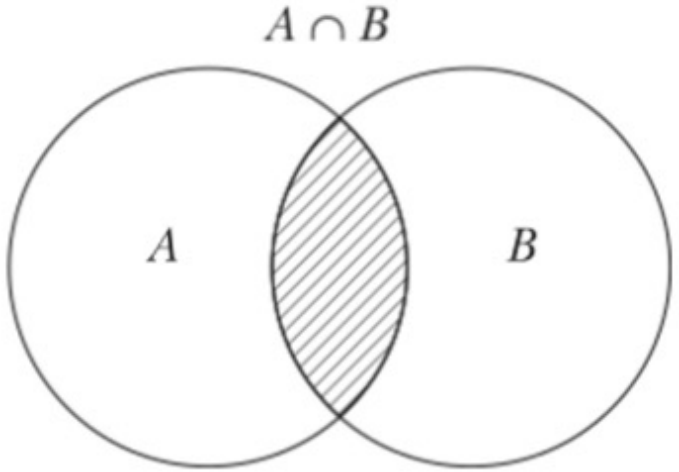
ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21

Отношение B

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1896	Галкин	Иваново	40

A ∩ B

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45



Реляционная алгебра
Разность

Отношение A

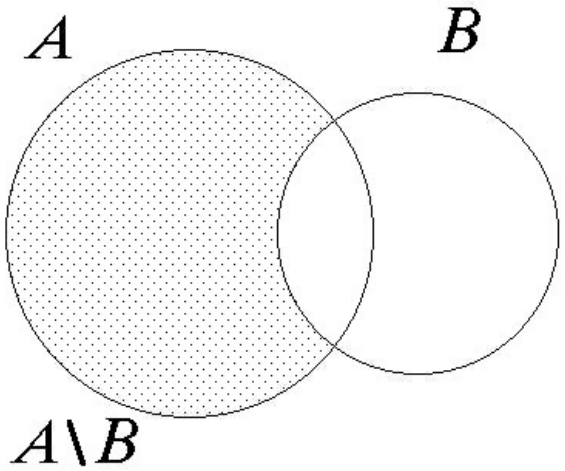
ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21

Отношение B

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1896	Галкин	Иваново	40

A – B

ID_NUM	NAME	CITY	AGE
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21



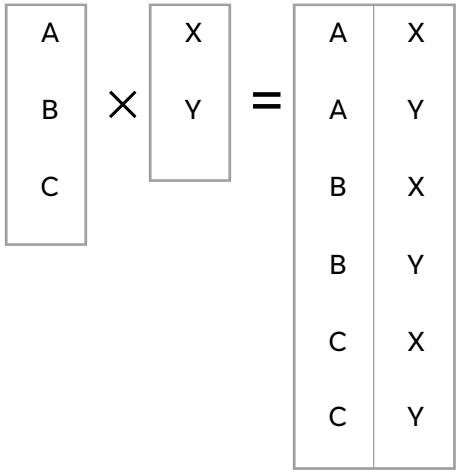
Реляционная алгебра
Декартовое произведение

Отношение A

S1
S2
S3

Отношение B

P1
P2
P3
P4



A × B

S1	P1
S1	P2
S1	P3
S1	P4

S2	P1
S2	P2
S2	P3
S2	P4

S3	P1
S3	P2
S3	P3
S3	P4

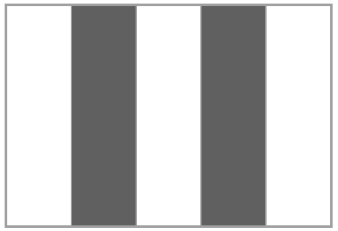
Кардинальность?
Степень?

Реляционная алгебра

Проекция

Отношение A.

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21
1896	Галкин	Москва	30



A [NAME, CITY]

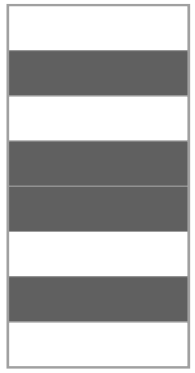
NAME	CITY
Иванов	Москва
Петров	Нижний Новгород
Сидоров	Рязань
Галкин	Москва

A [CITY]

CITY
Москва
Нижний Новгород
Рязань

Реляционная алгебра

Выборка



Отношение A.

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21
1896	Галкин	Москва	30

A where CITY = 'Москва'

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1896	Галкин	Москва	30

A where CITY = 'Москва' and AGE < 40

ID_NUM	NAME	CITY	AGE
1896	Галкин	Москва	30

Реляционная алгебра

Деление

Отношение A

S#	P#
S1	P1
S1	P2
S1	P3
S1	P4
S2	P1
S2	P3
S3	P2
S3	P3

Отношение B

P#
P1
P2
P3

Отношение B1

P#
P1
P2
P3
P4

Отношение B2

P#
P1
P2
P3

A/B

S1
S2
S3

A/B1

S1
S2
S3

A/B2

S1
S2

Реляционная алгебра

Деление

Отношение A

S#	P#
S1	P1
S1	P2
S1	P3
S1	P4
S2	P1
S2	P3
S3	P2
S3	P3

Отношение B

P#
P1
P2

Отношение B1

P#
P2
P3

Отношение B2

P#
P1
P2
P3

A/B

S1
S2

A/B1

S1
S3

A/B2

S1

Реляционная алгебра
Соединение

Отношение A (поставщики)

ID_NUM	NAME	CITY	STATUS
1809	Иванов	Москва	20
1996	Петров	Нижний Новгород	15
1777	Сидоров	Рязань	10

Отношение B (поставки)

ID_NUM	IP_NUM	QTY
1809	P123	100
1809	P896	200
1777	P432	150
1996	P432	150
1996	P123	250

(A×B (RENAME ID_NUM AS AID_NUM) WHERE QTY <200

ID_NUM	NAME	CITY	STATUS	AID_NUM	IP_NUM	QTY
1809	Иванов	Москва	20	1809	P123	100
1996	Петров	Нижний Новгород	15	1996	P432	150
1777	Сидоров	Рязань	10	1777	P432	150

Реляционная алгебра

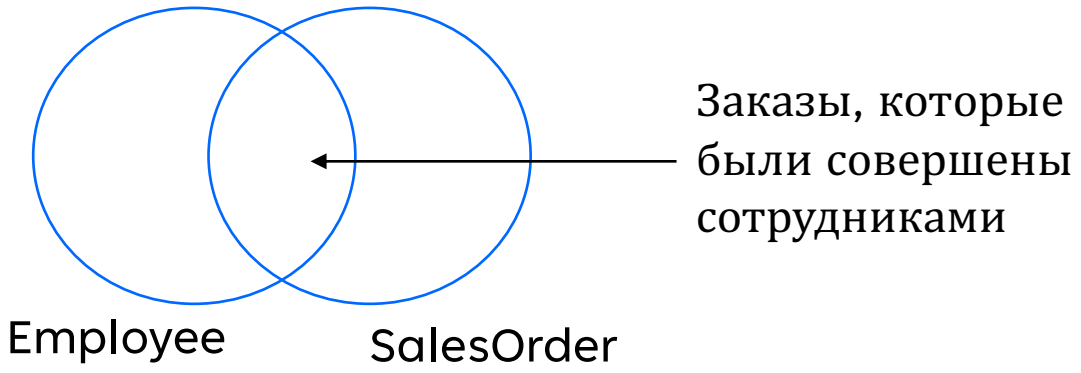
Операции реляционной алгебры в SQL

объединение	**UNION** [ALL] [CORRESPONDING [BY {Имя_столбца[, ...]}]]
пересечение	**INTERSECT** [ALL] [CORRESPONDING [BY {Имя_столбца[, ...]}]]
разность	**EXCEPT** [ALL] [CORRESPONDING [BY {Имя_столбца[, ...]}]]
произведение	FROM { <Источник_данных> } [,...,n] == **CROSS JOIN** **<Источник_данных> ::= <имя_таблицы>**
соединение	FROM { <Источник_данных> } [,...,n] **<Источник_данных> ::= <связка_таблиц>** **<тип_связывания> ::=** **[INNER | {{LEFT | RIGHT | FULL }[OUTER] }] JOIN**
проекция	**SELECT DISTINCT** <Список_выбора>
выборка	**WHERE** <условие_отбора>

3 Запросы к нескольким таблицам с соединениями

Соединения (JOIN).
Основные концепции

- Соединяет строки из нескольких таблиц с помощью указываемого критерия соответствия:
  - Обычно базируется на парах первичный ключ – внешний ключ
  - Например, вернуть строки, которые соединяют данные из таблиц **Employee** и **SalesOrder** на основе равенства значений первичного ключа **Employee.EmployeeID** и внешнего **SalesOrder.EmployeeID**
- Можно представлять таблицы в виде множеств на диаграммах Венна



Соединения (JOIN).
Синтаксис соединений

- **ANSI SQL-92**
  - Таблицы соединяются при помощи оператора JOIN в предложении FROM
    - Предпочитаемый вариант

```
SELECT ...
FROM   Table1 JOIN Table2
       ON <on_predicate>;
```

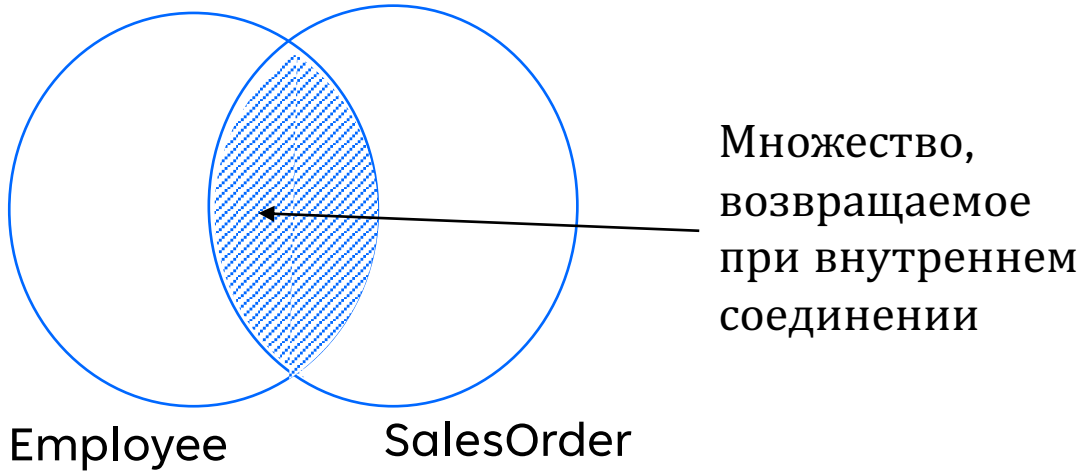
- **ANSI SQL-89**
  - Таблицы соединяются при помощи перечисления через запятую в предложении FROM
    - Не рекомендуется: при ошибке возможно декартово произведение!

```
SELECT ...
FROM   Table1, Table2
WHERE  <where_predicate>;
```

Соединения (JOIN).
Внутренние соединения ([Inner] Joins)

- Возвращает только те строки, для которых найдено соответствие в обоих входных таблицах
- Соответствие между строками таблиц через указанные в предикате атрибуты
- Если используется операция =, то получается эквисоединение (equi-join)

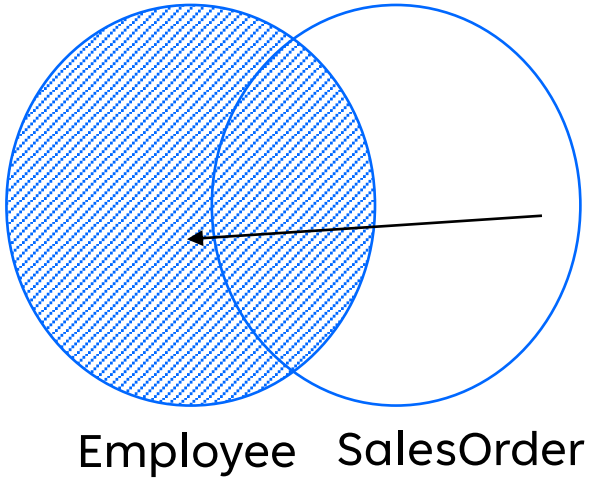
```
SELECT emp.FirstName, ord.Amount
FROM HR.Employee AS emp
[INNER] JOIN Sales.SalesOrder AS ord
ON emp.EmployeeID = ord.EmployeeID
```



Соединения (JOIN).
Внешние соединения([Outer] Joins)

- Возвращает все строки из одной таблицы и найденные соответствия из другой
- Все строки одной из таблиц сохраняются
  - Обозначается ключ. словами: LEFT, RIGHT, FULL
  - Все строки из “сохраненной” таблицы попадают в результат
- Извлекаются соответствия из другой таблицы
- Дополнительные строки добавляются к результату, если соответствие не найдено
  - Используются значения NULL для столбцов, где не найдено соответствие

```
SELECT emp.FirstName, ord.Amount
FROM HR.Employee AS emp
LEFT [OUTER] JOIN Sales.SalesOrder AS ord
ON emp.EmployeeID = ord.EmployeeID;
```



Множество, возвращаемое левым внешним соединением

Перекрестные соединения (Cross Join)

- Комбинирует каждую строку из первой таблицы с каждой строкой из второй табл.
- Выводятся все возможные комбинации
- Логическая основа для внутренних и внешних соединений
  - Внутреннее соединение начинает с декартова произведения, потом накладывается фильтр
  - Внешнее соединение после фильтрации результатов декартова произведения добавляет NULL для строк без соответствия

Employee	
EmployeeID	FirstName
1	Dan
2	Aisha

Product	
ProductID	Name
1	Widget
2	Gizmo

```
SELECT emp.FirstName, prd.Name
FROM HR.Employee AS emp
CROSS JOIN Production.Product AS prd;
```

Result	
FirstName	Name
Dan	Widget
Dan	Gizmo
Aisha	Widget
Aisha	Gizmo

Самосоединение (Self Joins)

- Сравнивает строки в той же таблице
- Создает два экземпляра той же таблицы в предложении FROM
  - Требуется, как минимум, один псевдоним

```
SELECT emp.FirstName AS Employee,
       man.FirstName AS Manager
FROM HR.Employee AS emp
LEFT JOIN HR.Employee AS man
ON emp.ManagerID = man.EmployeeID;
```

Employee		
EmployeeID	FirstName	ManagerID
1	Dan	NULL
2	Aisha	1
3	Rosie	1
4	Naomi	3

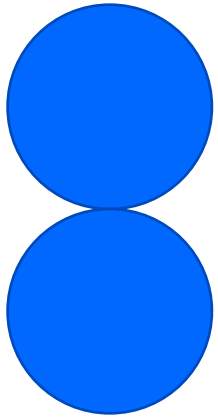
Result	
Employee	Manager
Dan	*NULL*
Aisha	Dan
Rosie	Dan
Naomi	Rosie

4 Теоретико-множественные операторы

Запросы с объединением (UNION)

- UNION возвращает набор уникальных строк, собранных из обоих SELECT-операторов
- UNION устраняет дубликаты при обработке запросов (влияет на производительность)
- UNION ALL сохраняет дубликаты при обработке запросов

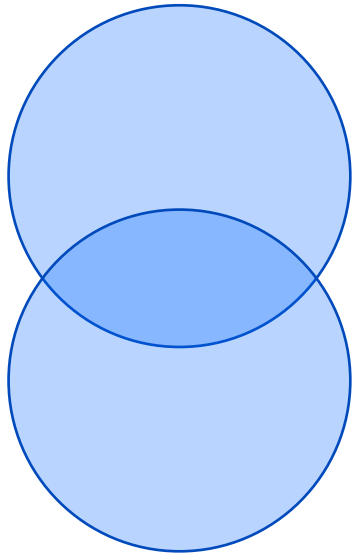
```
-- возвращаются только уникальные (несовпадающие)
-- строки из обоих запросов
SELECT CountryRegion, City FROM SalesLT.Employees
UNION
SELECT CountryRegion, City FROM SalesLT.Customers;
```



Запросы с пересечением (INTERSECT)

INTERSECT возвращает только те уникальные строки, которые присутствуют в обоих запросах

```
-- только строки, одновременно присутствующие в обоих запросах
SELECT CountryRegion, City FROM SalesLT.Employees
INTERSECT
SELECT CountryRegion, City FROM SalesLT.Customers;
```

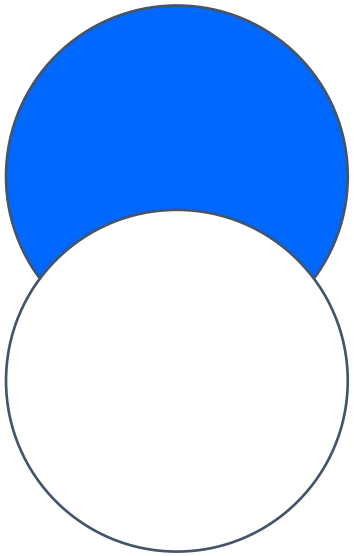


Запросы с разностью (EXCEPT)

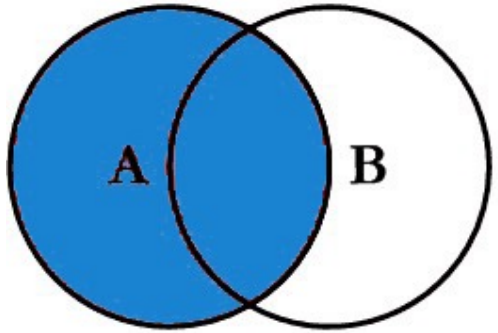
EXCEPT возвращает только уникальные строки, которые присутствуют в первом запросе, но отсутствуют во втором.

- Порядок указания запросов имеет значение

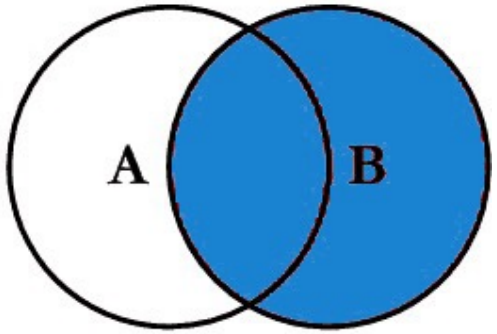
```
-- будут возвращены только строки из Employees
SELECT CountryRegion, City FROM SalesLT.Employees
EXCEPT
SELECT CountryRegion, City FROM SalesLT.Customers;
```



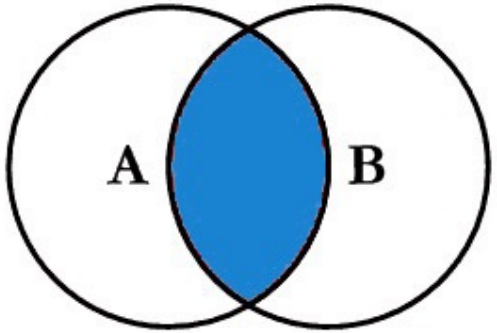
SQL JOINS



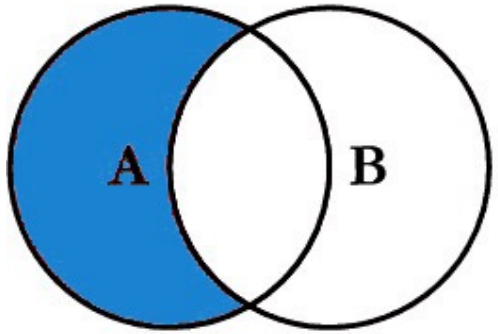

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



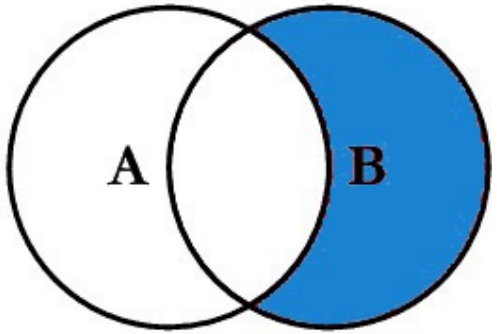

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



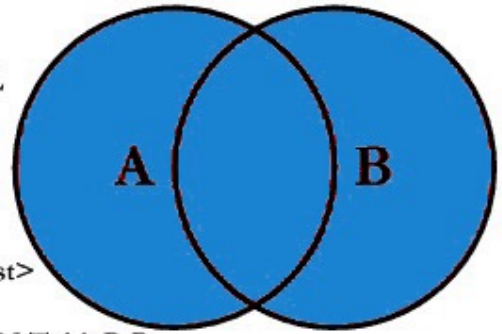

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



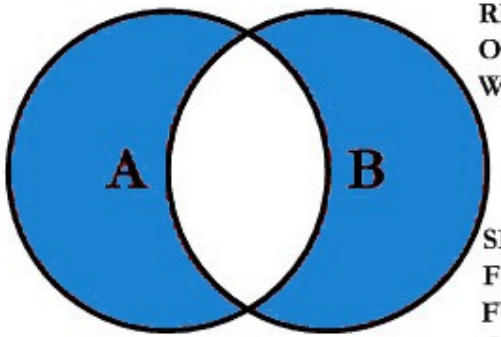

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```